Enterprise Operations Copilot

Enterprise Operations Copilot is a grounded document intelligence application designed to help users search, interpret, and answer questions from a curated enterprise knowledge base. The system is built to support operational use cases across internal business documents such as (SOPs), escalation policies, sales-to-operations handoff guides, implementation checklists, FAQs, and CRM process references. Instead of relying on open-web information, the copilot retrieves relevant content from indexed enterprise documents and generates answers only from that retrieved context.

The project was developed as a practical Retrieval-Augmented Generation (RAG) MVP to demonstrate how cloud-based retrieval and local language models can work together in a real business setting. Azure Blob Storage and Azure AI Search are used to store, index, and retrieve relevant documents, while a local large language model running through Ollama generates grounded answers from the top search results. The frontend is built in Streamlit to provide a clean and simple user experience with a query interface, grounded answer section, source transparency, and supporting search results. The main goal of Enterprise Operations Copilot is to simulate how an internal enterprise assistant could help teams quickly access operational knowledge without manually searching across multiple documents.

Project Key Details

Field	Details
Project Type	Portfolio-ready retrieval-augmented generation (RAG) MVP
Primary Use Case	Grounded enterprise document search and answer generation
Core Stack	Azure Blob Storage, Azure AI Search, Streamlit, Ollama (gemma3:4b)
Current Status	Working MVP with grounded answers, sources, and supporting search results

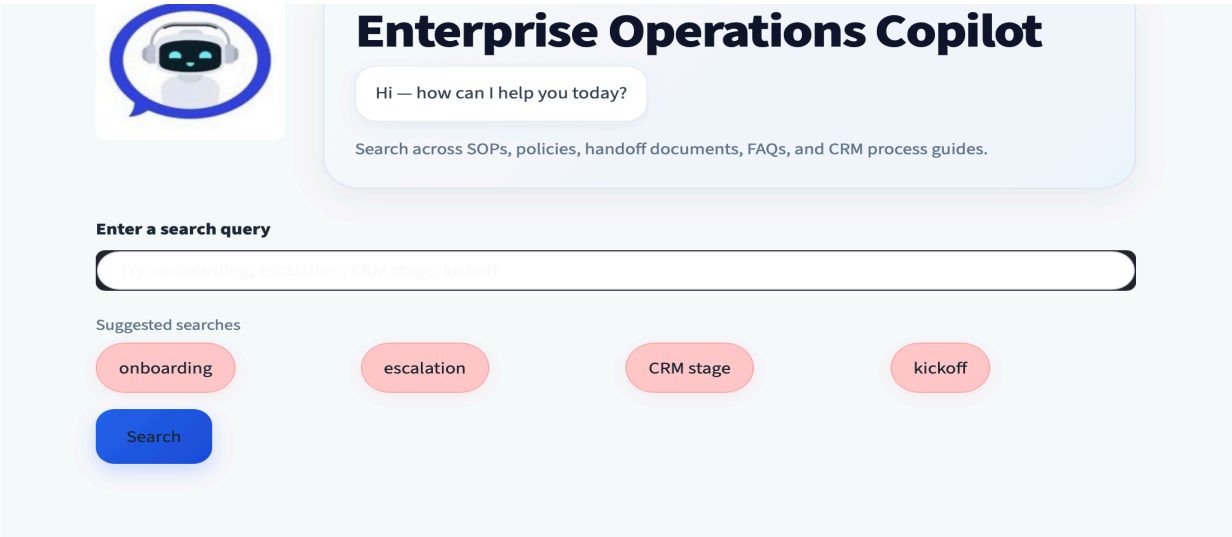


Figure 1. Cover screenshot

1. Project Overview

Enterprise Operations Copilot is a document-grounded assistant built to answer operational questions from a curated enterprise knowledge base. The system indexes SOPs, policies, handoff documents, FAQs, implementation checklists, and CRM process guidance, retrieves relevant passages through Azure AI Search, and generates grounded answers through a local large language model running in Ollama.

The current version is intentionally scoped as a single-turn grounded Q&A MVP rather than a full multi-turn chat product. This keeps the implementation stable, reduces hallucination risk, and provides a strong portfolio artifact that clearly demonstrates retrieval, answer generation, UI integration, and product thinking.

2. Solution Objectives

- Create a searchable knowledge base from enterprise-style PDF and DOCX documents.
- Build a clean user interface for asking operational questions.
- Generate answers grounded only in the indexed documents.
- Show the supporting sources used for each answer.
- Package the project so it can be published to GitHub as a polished MVP.

3. Architecture

Layer	Tool / Service	Purpose	Notes
Storage	Azure Blob Storage	Stores the knowledge base documents	Contains SOPs, policies, FAQs, handoff process files, and reference guides
Retrieval	Azure AI Search (Free tier)	Indexes documents and retrieves the most relevant passages	Provides the top matching records passed into the answer-generation step
Generation	Ollama with gemma3:4b	Creates grounded answers using retrieved context only	Runs locally to avoid Azure OpenAI access and cost constraints
Application UI	Streamlit	Presents the search interface, answer card, sources, and search results	Delivers the working MVP experience for demonstrations and GitHub screenshots

End-to-end flow: user question → Azure AI Search retrieves relevant documents → top results are converted into context → Ollama generates a grounded answer → the app displays the answer, source titles, and supporting search results.

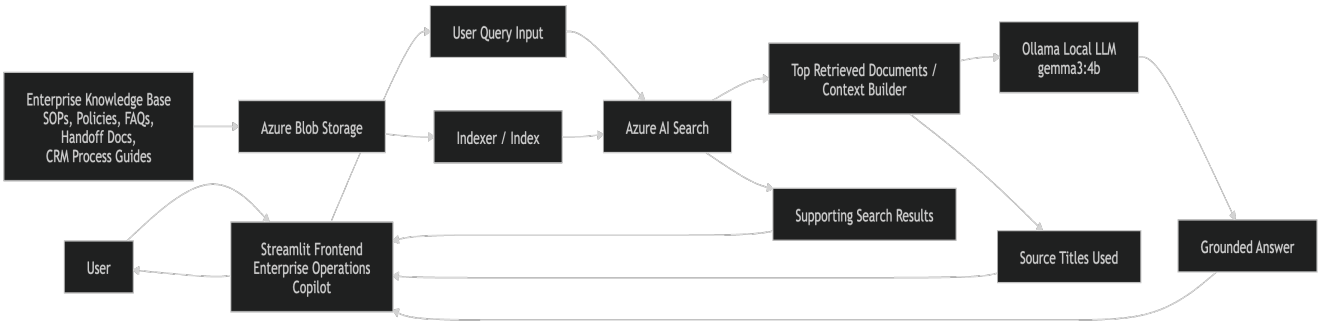
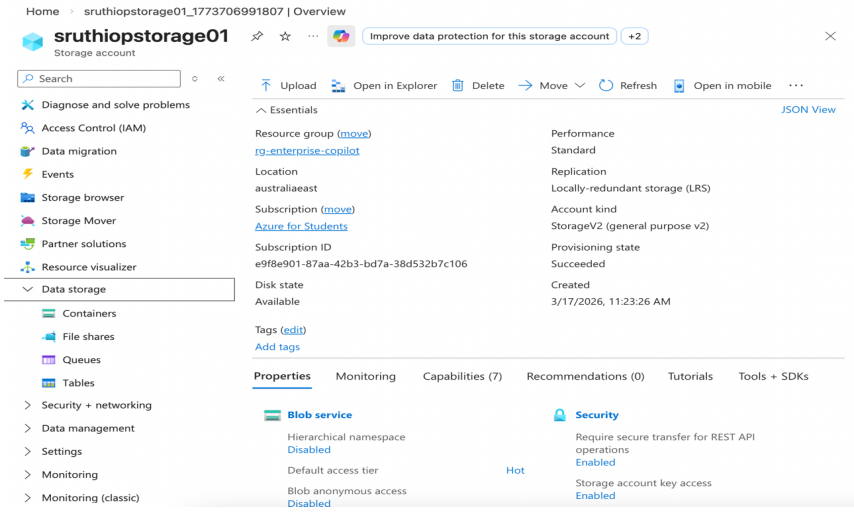


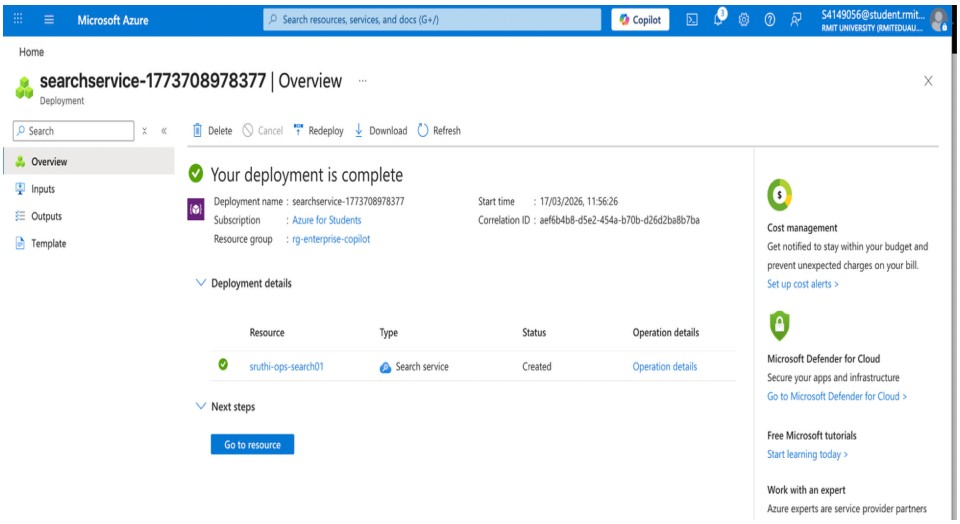
Figure 2. System architecture

4. Implementation Phases

Phase 1 – Infrastructure and Indexing

Created the Azure resource group, storage account, blob container, and Azure AI Search service. Uploaded the knowledge-base documents, configured the indexer, and verified successful ingestion.





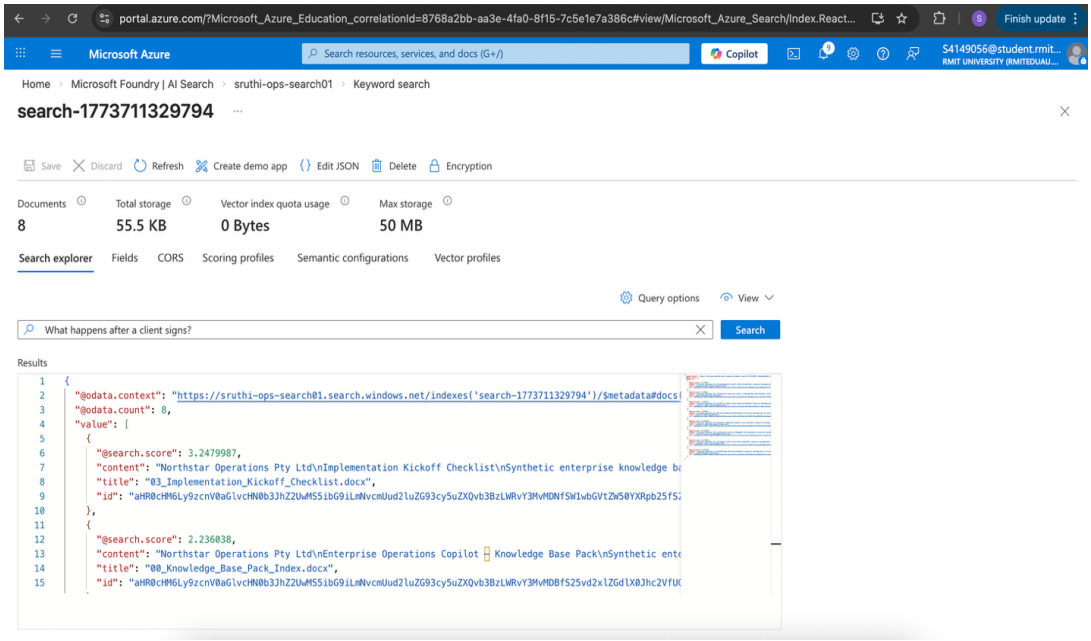
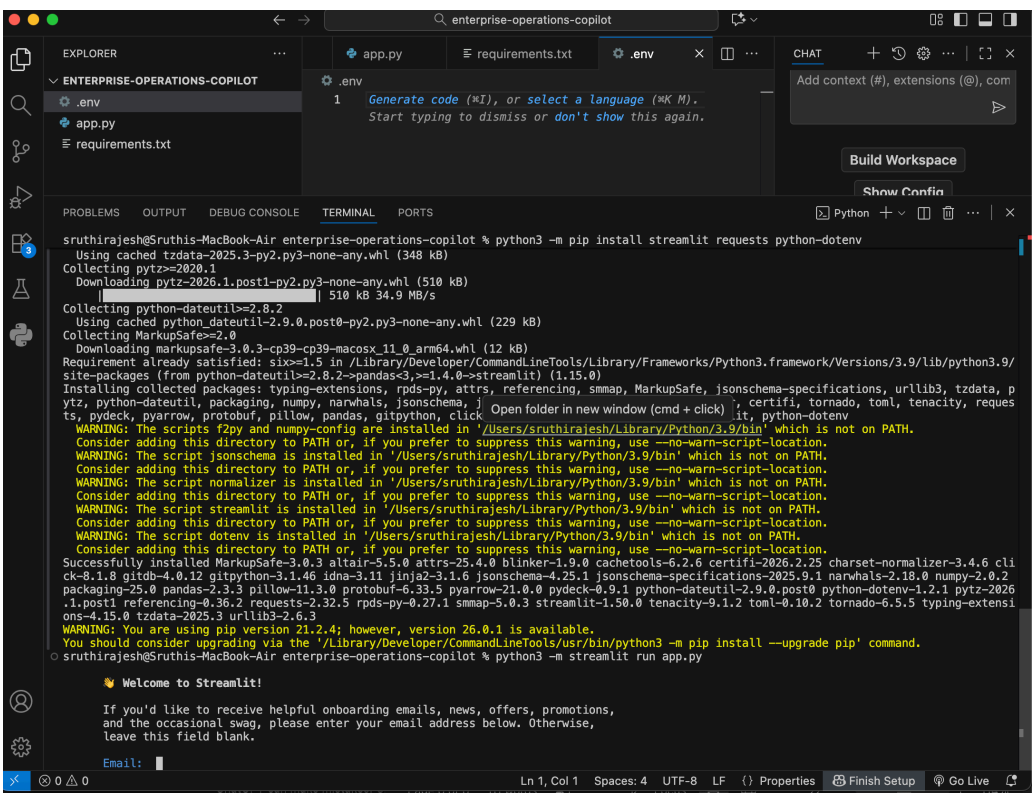


Figure 3. Azure resource, storage and AI Azure Search Service

Phase 2 – Frontend Retrieval App

Built the Streamlit interface with a branded hero section, search box, suggestion chips, styled search button, result cards, and expanded excerpt view. The app successfully queried Azure AI Search and returned relevant document matches.



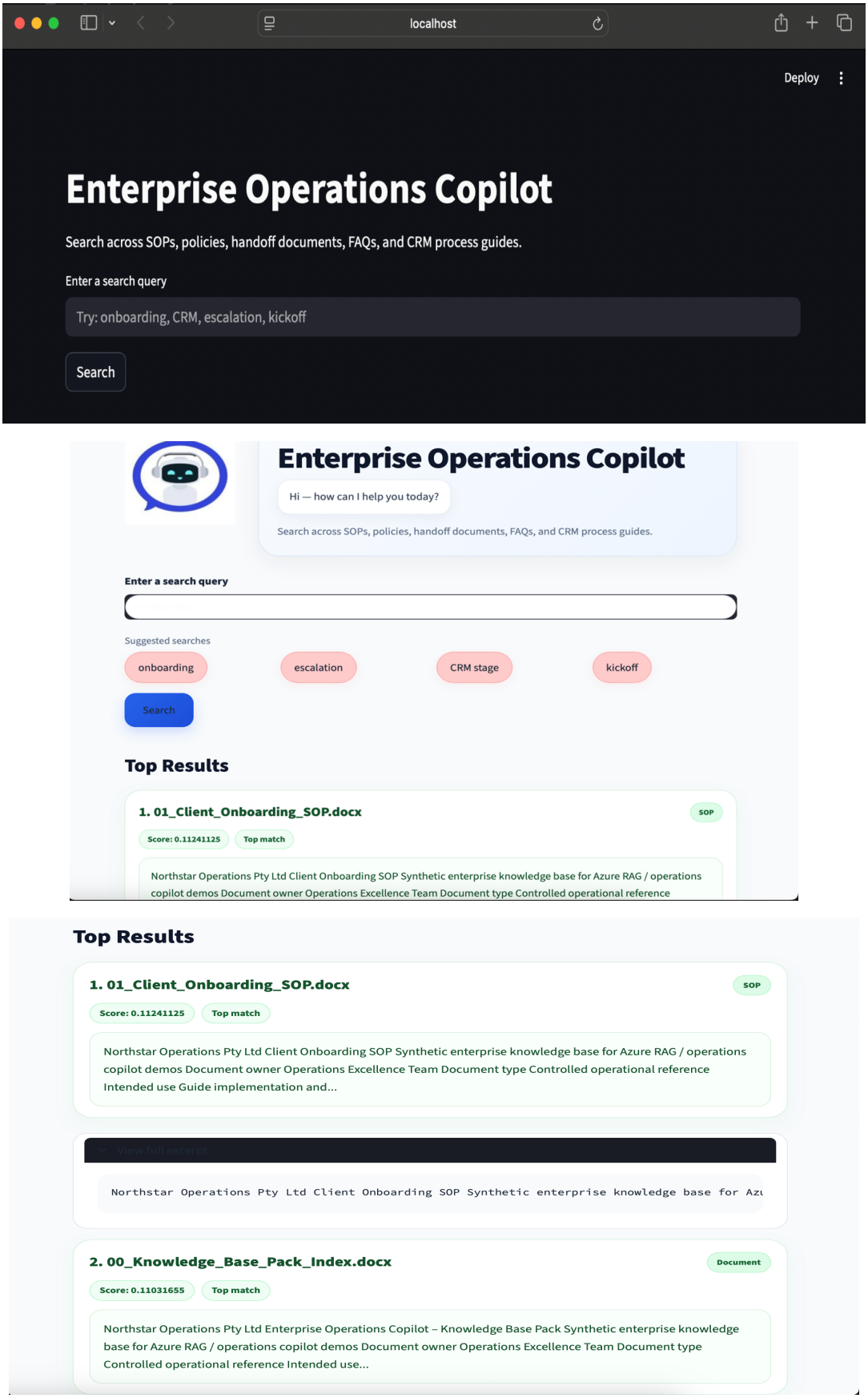
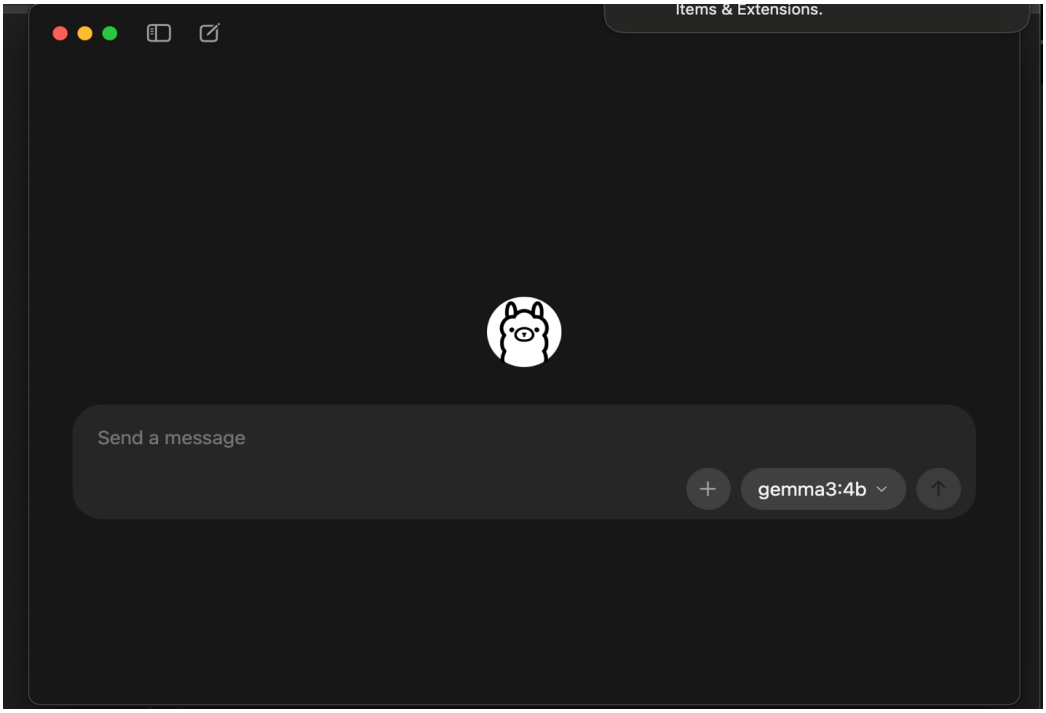
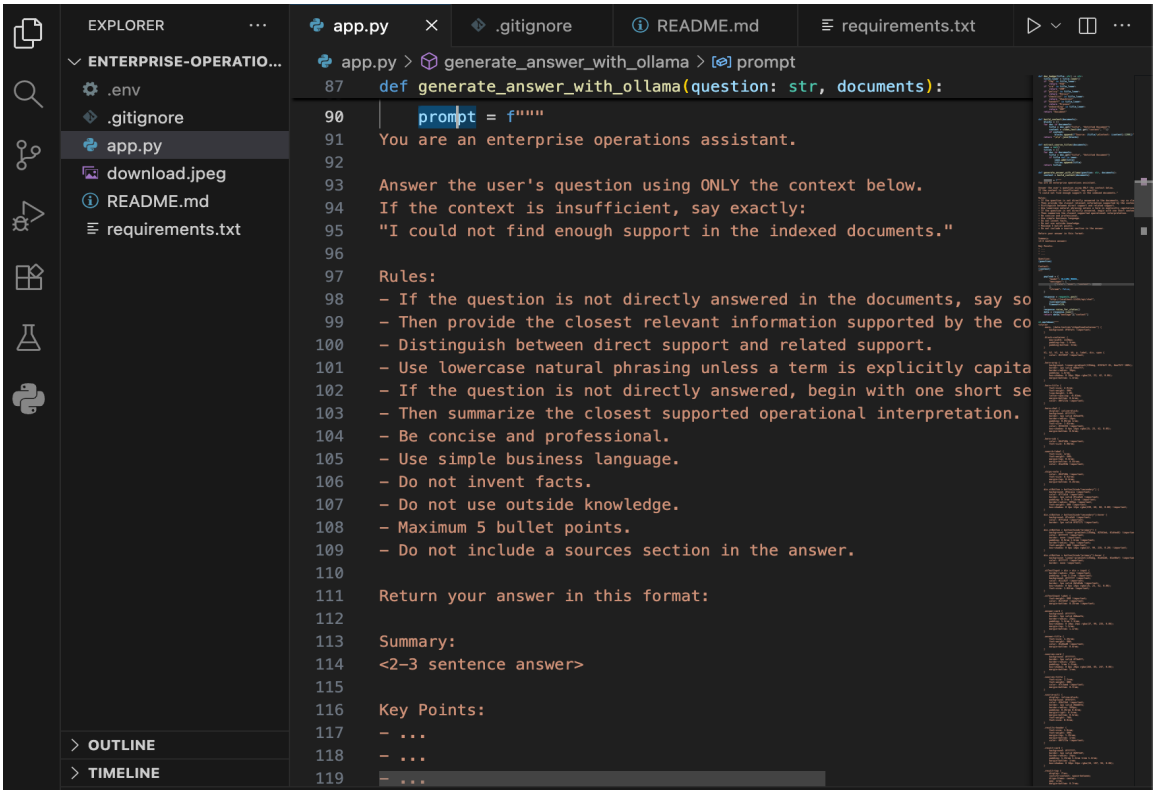


Figure 4. App successfully captures AI Azure search service

Phase 3 – Grounded Answer Generation

Integrated a local Ollama model on the developer machine. Retrieved results are transformed into context and passed to the model with prompt constraints so answers remain grounded in the indexed documents. The UI displays a grounded answer plus source documents before the underlying search results.





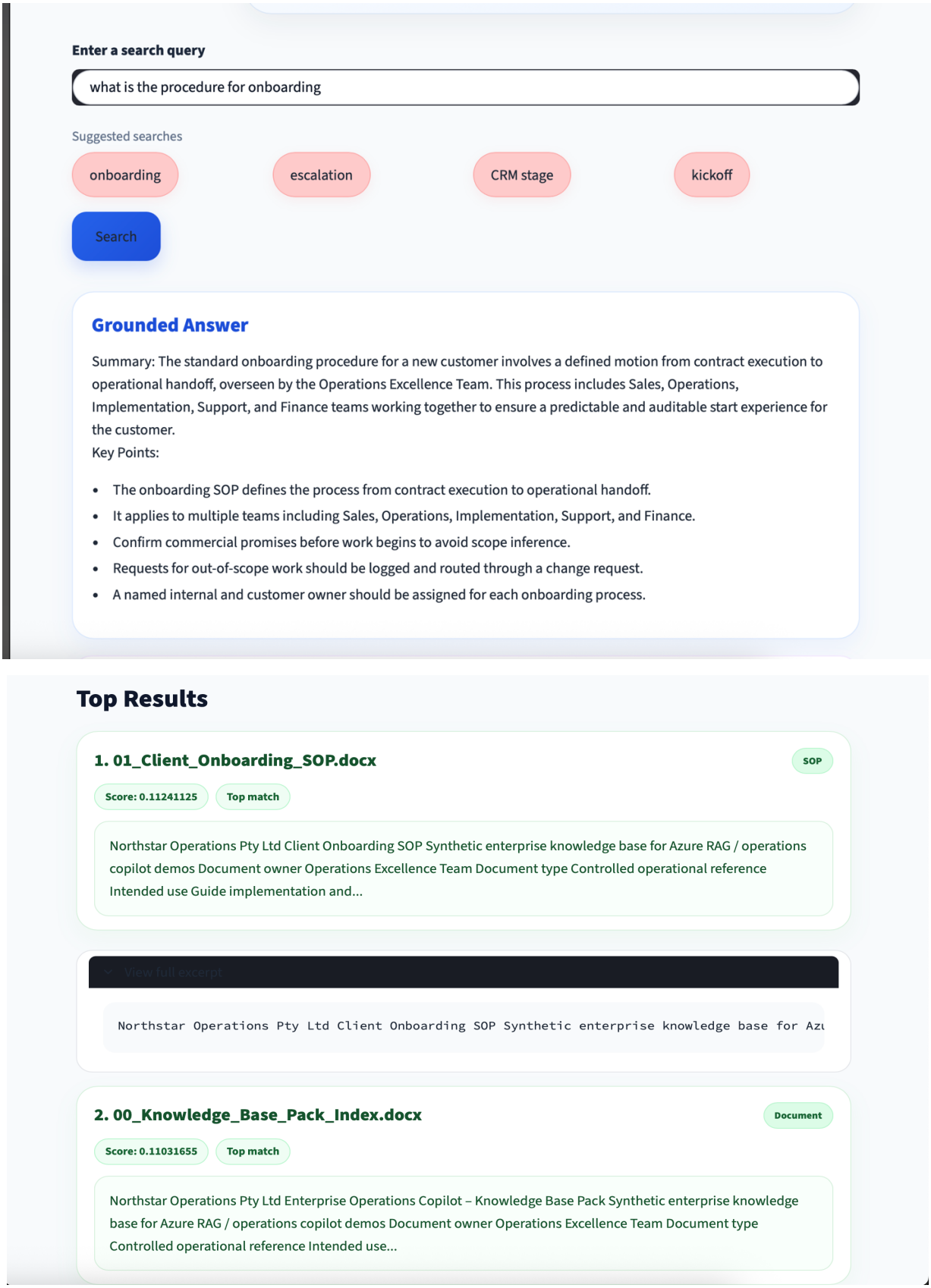


Figure 6. Using the given prompt, integrating local Ollama, UI displays grounded answer and the documents

5. Current MVP Features

Feature	Description
Search interface	Single search box with suggested prompt chips for common operational queries
Grounded answer section	Displays a model-generated answer based only on indexed documents
Sources used	Shows source document titles as pills for transparency
Supporting search results	Displays top retrieved documents with score, badge, and excerpt
Expandable evidence	Lets the user open a fuller excerpt from each retrieved document
Prompt guardrails	Tells the model to avoid outside knowledge and distinguish direct support from related support

6. Local Setup and Run Instructions

1. Clone the repository or open the project folder in VS Code.
2. Create a .env file containing AZURE_SEARCH_ENDPOINT, AZURE_SEARCH_KEY, and AZURE_SEARCH_INDEX.
3. Ensure Ollama is installed locally and the selected model is available.
4. Install Python dependencies from requirements.txt.
5. Run the application with python3 -m streamlit run app.py.

AZURE_SEARCH_ENDPOINT=https://<your-search-service>.search.windows.net

AZURE_SEARCH_KEY=

AZURE_SEARCH_INDEX=<your-index-name>

7. GitHub Repository Structure

Path	Purpose
app.py	Main Streamlit application file
requirements.txt	Python dependencies for setup and execution
.env.example	Template showing required environment variable names only
assets/	Screenshots, chatbot image, and demo visuals for the README
docs/Enterprise_Operations_Copilot_Project_Documentation.docx	Detailed project document for GitHub and portfolio use

README.md	Public-facing project overview, setup steps, architecture, screenshots, and sample questions
.gitignore	Excludes .env, caches, virtual environments, and local-only files

9. Future Enhancements

- Add multi-turn follow-up memory using Streamlit session state and short conversation history
- Add inline evidence markers or document section references
- Deploy a hosted model or production-safe inference path for broader sharing
- Deploy the frontend to Streamlit Community Cloud or another web host
- Add semantic reranking, query rewriting, or chunk optimization

12. Conclusion

The Enterprise Operations Copilot is now a working RAG MVP that combines Azure-based retrieval with local grounded answer generation. The current build is strong enough for GitHub publication and portfolio use because it demonstrates end-to-end system setup, document indexing, application development, prompt design, source transparency, and practical product scoping.

Follow-up conversation memory can be added later as a deliberate enhancement rather than as a dependency for the current MVP. For portfolio purposes, the current version already communicates strong technical judgment, product discipline, and a clear understanding of grounded AI application design.